

Padrões de Design

AI-Específicos (PT-5)

Arquitetura de Sistemas com IA · Módulo 4.5

Prof. Ahirton Lopes, Ph.D.

Os Quatro Padrões, Um Gateway Só

01

RAG (Módulo 4.1)

Multi-Index roteia pro domínio certo, Hybrid Search combina BM25 com embedding real, e Agentic RAG insiste com estratégia mais ampla se a confiança vier baixa — vira o sinal do próximo passo.

02

Roteamento (Módulo 4.2)

Intent-Based Routing classifica a intenção primeiro; Model Router decide, com base nela, se o modelo barato basta ou se precisa do caro.

03

Cache e Streaming (Módulo 4.3)

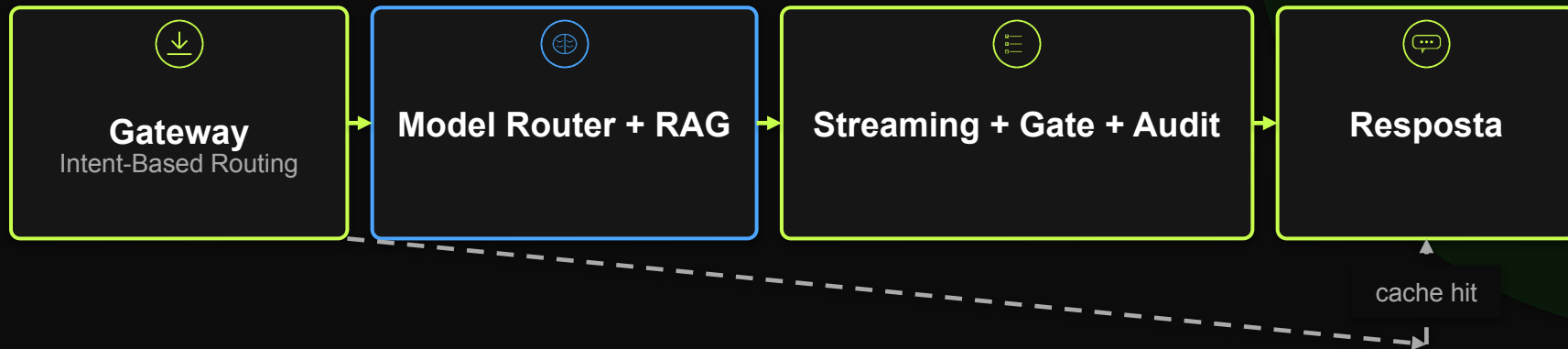
Semantic Cache evita reprocessar pergunta parecida; a resposta que precisa ser gerada aparece token a token, não tudo de uma vez.

04

Approval Gate (Módulo 4.4)

Confiança baixa ou síntese de CSR aciona o Approval Gate; toda decisão — aprovada, rejeitada, ou nem chegou a escalar — vai pra trilha de auditoria.

TrialForge: O Fluxo Ponta a Ponta



TrialForge: O Gateway em JavaScript

Os quatro padrões numa função só, rodando localmente com Ollama

```
async function processarRequisicao(pergunta) {  
  const intencao = classificarIntencao(pergunta);  
  const embedding = await embedar(pergunta);  
  const cache = consultarCache(embedding);  
  if (cache.similaridade >= LIMIAR_CACHE) return cache.entrada.resposta;  
  const modelo = intencao === 'síntese_csr' ? MODELO_CARO : MODELO_BARATO;  
  const { clausula, confianca } = await buscarClausulaAgentica(pergunta, embedding, indice);  
  const stream = await ollama.chat({ model: modelo, messages, stream: true });  
  for await (const parte of stream) process.stdout.write(parte.message.content);  
  if (intencao === 'síntese_csr' || confianca < LIMIAR_CONFIANCA) {  
    aprovado = await pedirAprovacaoHumana(rascunho);  
  }  
  registrarAuditoria({ pergunta, intencao, modelo, confianca, aprovado });  
}
```

Roda de graça na sua máquina — troque só `ollama.chat()/embed()` pra usar Claude, Gemini ou GPT.

Calibração real medida: 0,825 (paráfrase) · 0,667 (síntese CSR) · 0,633 (tema diferente) → limiar final 0,75

RAG (Multi-Index/Hybrid/Agentic) real: protocolo converge na 1ª tentativa (0,848) · tema fora do banco esgota as 3 (0,633)

Missão Prática #04

01

Mapeie os 4 padrões no seu contexto

Para uma tarefa real do seu trabalho, identifique: qual seria a intenção, qual modelo processaria, o que seria cache-ável, e o que justificaria um Approval Gate.

02

Calibre um limiar com dado real

Não assuma um número de limiar de similaridade ou confiança — teste contra pares de pergunta parecida e pergunta diferente, do jeito que fizemos neste vídeo, e documente o valor encontrado.

03

Rode o protótipo

Execute o Gateway na sua máquina (entrega oficial em JavaScript; Python é só referência) e registre os 5 casos: cache miss, cache hit, os 2 Approval Gate, e o acerto de primeira.

Entrega: mapeamento dos 4 padrões + limiar calibrado com evidência + log da execução, num único arquivo.

Quatro Padrões, Testados. E Numa Escala Maior?

O Gateway de hoje resolve uma requisição de cada vez, para um estudo clínico. Mas a Vitalis Pharma roda dezenas de estudos ao mesmo tempo, em múltiplos países. O Módulo 4 está completo — RAG, roteamento, cache e streaming, e o Approval Gate formalizado.

Módulo 5 → Arquitetura Enterprise: multi-tenant, custo em escala, e governança quando o sistema deixa de ser um protótipo e vira produto.